ANNA UNIVERSITY: CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this mini project report

"SMART SOLAR TRACKING & MONITORING SYSTEM"

is the bonafide work of **BHUVANESHWARI M (312423105010) ,KAVINRAGAVINYAA KR (312423105033), SUBHA S R E E B (312423105059) , DEEPALAKSHMI A (312423105012)** who carried out the Mini project work under my supervision.

SIGNATURE	**SIGNATURE**
Dr.D Kirubakaran M.Tech., Ph.D.,	Guide Name
Professor	Assistant Professor
HEAD OF THE DEPARTMENT	**SUPERVISOR**
Department of Electrical and Electronics Engineering	Department of Electrical and Electronics Engineering
St.Joseph's Institute of Technology	St.Joseph's Institute of Technology
Old Mamallapuram Road	Old Mamallapuram Road
Chennai-600119	Chennai-600119

Submitted for the Viva-Voce held on _____

(INTERNAL EXAMINER)

CERTIFICATE OF EVALUATION

College Name : St. Joseph's Institute of Technology

Branch & Semester : Electrical and Electronics Engineering

S.NO	NAMES OF STUDENTS	TITLE OF THE MINI PROJECT	NAME OF THE SUPERVISOR
1.	BHUVANESHWARI M (312423105010)	"SMART SOLAR TRACKING & MONITORING SYSTEM"	Staff Name
2.	DEEPALAKSHMI A (312423105012)		Designation
3.	KAVINRAGAVINYAA KR (312423105033)		
4.	SUBHA SREE B (312423105059)		

The report of the mini project work submitted by the above students for Mini Project in Electrical and Electronics Engineering of Anna University were evaluated and confirmed to be reports of the work done by the above students and then evaluated.

ABSTRACT

This project integrates a solar tracking mechanism with a solar power monitoring system. A light-seeking tracker (two pairs of LDRs) adjusts the solar panel orientation using servo motors to maximize sun exposure. Simultaneously, an ESP32 measures panel voltage and current (via a voltage divider and ACS712), computes instantaneous power and accumulated energy, displays local data on an OLED, and uploads time-stamped readings to ThingSpeak for remote monitoring. The tracker increases energy harvest by keeping the panel normal to sunlight, while the monitoring subsystem quantifies the improvement. The system is low-cost, suitable for lab-scale solar panels, and demonstrates practical concepts in power electronics, sensor interfacing, control and IoT.

INTRODUCTION

The growing demand for sustainable and renewable energy has accelerated the adoption of solar power systems worldwide. However, the efficiency of a solar panel is highly dependent on its orientation to the sun throughout the day. A fixed-position panel cannot capture the maximum possible sunlight, which reduces its overall energy output. To address this limitation, solar tracking systems are employed to automatically adjust the panel's position to follow the sun's movement.

This project, titled "Smart Solar Tracking and Monitoring System," aims to enhance the efficiency of solar energy collection by integrating a light-based tracking mechanism with real-time power monitoring capabilities. The system uses Light Dependent Resistors (LDRs) to detect sunlight direction and automatically rotates the panel using servo motors to align it for optimal exposure. In addition, the system monitors key electrical parameters—such as voltage, current, and power output—using sensors connected to a microcontroller (ESP32). These parameters are displayed on a local OLED screen and also uploaded to an IoT platform (ThingSpeak) for remote access and analysis.

By combining solar tracking with IoT-based monitoring, the project not only increases energy generation efficiency but also provides valuable insights into system performance. This makes it a practical and scalable solution for students, researchers, and developers interested in smart renewable energy system

OBJECTIVES

- Implement a solar tracking algorithm (two-axis or single-axis) using LDR sensors to steer the panel toward maximum sunlight.
- Measure solar panel voltage, current, power and cumulative energy.
- Display live data locally (OLED) and upload to ThingSpeak for remote visualization.
- Compare energy harvested with and without tracking (simple experiment).
- Keep system modular and safe for small lab panels.

SYSTEM OVERVIEW

Solar Panel mounted on tracker → (Mechanical frame + servo motors)
Sensors: LDR pairs for azimuth & elevation → ESP32 ADC
Power sensing: Voltage divider → ESP32 ADC; ACS712 current sensor → ESP32 ADC
Control: ESP32 calculates LDR difference → moves servos to reduce difference
Display & Cloud: OLED (I2C) + ESP32 WiFi → ThingSpeak
(Place a simple diagram image in your report showing these blocks & arrows — mechanical mount & servo axes, LDR pairs on panel corners, sensing & ESP32.)

HARDWARE (BOM) — APPROXIMATE INR

Item	**Qty**	**Approx. Cost (₹)**
ESP32 Development Board	1	450
Small Solar Panel (5–10 W)	1	800
ACS712 Current Sensor (20A or 5A)	1	250
Voltage Divider resistors (or voltage sensor module)	1	80

Item	Qty	Approx. Cost (₹)
2 × Micro Servos (SG90) **(for small panel)** or 2 × stronger servos	2	300 (150 each)
4 × LDR (Cadmium Sulphide photoresistor)	4	80
OLED 0.96" (I2C)	1	180
Metal/wood frame + mounting hardware	—	200
Diode (Schottky) + fuse + connectors	—	120
Wires, terminal blocks, perfboard, enclosure	—	200
Optional: Relay / SSR (for load connect/disconnect)	1	150
Estimated total		**~₹2,810**

Notes: If your panel is heavier than micro-servos can handle, use geared DC motors + motor drivers or larger servos (cost increases). For a demonstration/mini-panel, SG90 servos are fine.

MECHANICAL & SENSOR ARRANGEMENT

- **Tracker axes:**
- Single-axis (azimuth): rotate left-right — simpler. Use two LDRs left and right.
- Dual-axis (azimuth + elevation): 4 LDRs in quad (front-left, front-right, back-left, back-right) or two pairs (left-right, front-back), two servos (one rotates base, one tilts). I recommend **single-axis** for simplicity, or **dual-axis** if you want to show more advanced control.

 LDR mounting: Place LDRs at ends of a small shade to form pairs; add small shades to avoid direct glare (creates better differential reading).

 Panel mount: Attach panel to a lightweight platform fixed to servo horns; ensure servos are mounted firmly and have end limits to avoid over-rotation.

ELECTRICAL CIRCUIT & PIN MAP

Connections

- **LDRs (4) → voltage divider to ESP32 ADC pins**
  - LDR1 (left) voltage divider → ADC1_CH6 (GPIO34)
  - LDR2 (right) → ADC1_CH7 (GPIO35)
  - LDR3 (front) → ADC1_CH4 (GPIO32) *(optional for elevation)*
  - LDR4 (back) → ADC1_CH5 (GPIO33) *(optional)*
- **Voltage Measurement**
  - Solar panel positive → voltage divider (R1/R2) → ADC (GPIO36 / ADC1_CH0). Choose divider to map Vpanel_max to ≤3.0V.
- **Current Measurement**
  - Solar panel positive or load positive passes through ACS712 → ACS712 OUT → ADC (GPIO39 / ADC1_CH3). ACS712 Vcc → 5V, GND common.
- **OLED (I2C)**
  - SDA → GPIO21
  - SCL → GPIO22

- Vcc → 3.3V or 5V per module (check module)
- GND → GND

- **Servos**
  - Servo Azimuth PWM → GPIO18 (or any PWM-capable pin)
  - Servo Elevation PWM → GPIO19 (if dual-axis)
  - Servos Vcc → 5V supply (use separate 5V power if servos draw much current)
  - Servos GND → common GND with ESP32

- **Optional Relay**
  - Relay IN → GPIO23

SOFTWARE DESIGN & ALGORITHM

 Initialization: OLED, Wi-Fi, ThingSpeak API config, ADC pins, servo attach.

 LDR Read & Tracking Logic:

- For single-axis: read left and right LDR pair → compute difference diff = left - right.
- If abs(diff) > threshold → move servo a small step toward the brighter side (increase/decrease angle).
- Use hysteresis and small step increments to avoid oscillation. Limit servo angles to safe range (e.g., 0–170°).

 Monitoring:

- Read ADC for panel voltage and current; apply calibration to compute actual

V and I.

- Compute instantaneous power P = V × I. Update cumulative energy energy_Wh += P * (dt / 3600.0).

 Display & Upload: Update OLED every second; upload to ThingSpeak every 60 seconds (fields: V, I, P, energy, tracker_angle).

 Data Logging: Keep timestamps and optionally store daily totals in EEPROM

ESP32 CODE

```
/* IoT Solar Tracker + Power Monitor (ESP32, single-axis)
   - LDR left -> ADC_L, LDR right -> ADC_R
   - Voltage divider -> ADC_V
   - ACS712 OUT -> ADC_I
   - Servo -> servoPin (azimuth)
   - OLED on I2C
   - ThingSpeak upload
*/

#include <WiFi.h>
#include <HTTPClient.h>
#include <Wire.h>
#include <Adafruit_SSD1306.h>
#include <Servo.h>
```

```
// ====== User config ======
const char* ssid = "YOUR_SSID";
const char* password = "YOUR_PASS";
const char* thingspeak_api_key = "YOUR_THINGSPEAK_WRITE_KEY";

const int ADC_L = 34;   // LDR left
const int ADC_R = 35;   // LDR right
const int ADC_V = 36;   // panel voltage
const int ADC_I = 39;   // ACS712 output

const int servoPin = 18;

const float VREF = 3.3f;
const int ADC_MAX = 4095;

// Calibration (measure & set)
const float VOLT_DIV_FACTOR = 0.15f; // Vin = Vadc / VOLT_DIV_FACTOR
const float ACS_SENS_MV_PER_A = 66.0f; // mV/A typical (for 30A version)
const float ACS_SENS = ACS_SENS_MV_PER_A / 1000.0f; // V/A
float ACS_VOFFSET = VREF / 2.0f; // calibrate by reading at 0A

// Tracker params
```

```
int servoAngle = 90; // initial mid
const int SERVO_MIN = 0;

const int SERVO_MAX = 170;
const int STEP = 2; // degrees per adjustment
const int LDR_DIFF_THRESHOLD = 50; // raw ADC units, tune this
unsigned long lastTrackMove = 0;
const unsigned long TRACK_INTERVAL = 800; // ms between adjustments

// ThingSpeak
const char* thingspeak_url = "http://api.thingspeak.com/update";

// Globals
Adafruit_SSD1306 display(128, 64, &Wire);
Servo azServo;
unsigned long lastUpload = 0;
unsigned long lastLoop = 0;
float energy_Wh = 0.0;

void setup() {
  Serial.begin(115200);
  delay(1000);
  // OLED
  Wire.begin(21, 22);
  display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
  display.clearDisplay();
```

```
  // Servo
  azServo.attach(servoPin);

 azServo.write(servoAngle);

  // WiFi
  WiFi.begin(ssid, password);
  Serial.print("WiFi...");
  while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
  Serial.println("\nConnected.");

  // Calibrate ACS offset
  ACS_VOFFSET = analogRead(ADC_I) * VREF / ADC_MAX;
  Serial.print("ACS offset (V): "); Serial.println(ACS_VOFFSET, 3);
}

float readADCVoltage(int pin) {
  int raw = analogRead(pin);
  return (raw * VREF) / ADC_MAX;
}

float readPanelVoltage() {
  float vadc = readADCVoltage(ADC_V);
  return vadc / VOLT_DIV_FACTOR;
}
```

```
float readCurrent() {
  float vadc = readADCVoltage(ADC_I);
  float vdiff = vadc - ACS_VOFFSET;

  float current = vdiff / ACS_SENS; // A
  return current;
}

void uploadThingSpeak(float V, float I, float P, float E, int angle) {
  if (WiFi.status() != WL_CONNECTED) return;

  HTTPClient http;
  String url = String(thingspeak_url) + "?api_key=" + thingspeak_api_key +
             "&field1=" + String(V,2) + "&field2=" + String(I,3) +
             "&field3=" + String(P,2) + "&field4=" + String(E,3) +
             "&field5=" + String(angle);
  http.begin(url);
  int code = http.GET();
  Serial.println("TS code: " + String(code));
  http.end();
}

void loop() {
  unsigned long now = millis();
  // Read LDRs
  int ldrL = analogRead(ADC_L);
```

```
int ldrR = analogRead(ADC_R);

// Tracking (every TRACK_INTERVAL ms)
if (now - lastTrackMove > TRACK_INTERVAL) {

 int diff = ldrL - ldrR;
  if (abs(diff) > LDR_DIFF_THRESHOLD) {
    if (diff > 0) {
      // left brighter -> move left (reduce angle)
      servoAngle = max(SERVO_MIN, servoAngle - STEP);
    } else {
      // right brighter -> move right (increase angle)

      servoAngle = min(SERVO_MAX, servoAngle + STEP);
    }
    azServo.write(servoAngle);
  }
  lastTrackMove = now;
}

// Measurements
float Vpanel = readPanelVoltage();
float Ipanel = readCurrent();
float Power = Vpanel * Ipanel;

// Integrate energy
```

```
float dt = (now - lastLoop) / 1000.0f; // seconds
  if (lastLoop > 0) energy_Wh += Power * (dt / 3600.0f);
  lastLoop = now;

  // Display
  display.clearDisplay();
  display.setTextSize(1);
  display.setCursor(0,0);
  display.printf("V: %.2f V\n", Vpanel);
  display.printf("I: %.3f A\n", Ipanel);
  display.printf("P: %.2f W\n", Power);
  display.printf("E: %.3f Wh\n", energy_Wh);
  display.printf("S-Ang: %d\n", servoAngle);

  display.display();

  // Upload every 60s
  if (now - lastUpload > 60000) {
    uploadThingSpeak(Vpanel, Ipanel, Power, energy_Wh, servoAngle);
    lastUpload = now;
  }

  delay(200); // loop pace
}
```

Code notes & things to calibrate

- Set VOLT_DIV_FACTOR to match your resistor divider ratio. Example: if R1=56k (top), R2=10k (bottom), factor = R2/(R1+R2) ≈ 0.151. Then Vadc = Vin * factor, so Vin = Vadc / factor.
- Set ACS_SENS per your ACS712 module (e.g., 66mV/A → 0.066 V/A). Measure ACS offset at no-load to set ACS_VOFFSET.
- Tune LDR_DIFF_THRESHOLD and STEP to avoid hunting; add smoothing (moving average) to LDR readings for stability.
- If micro-servos jitter due to ADC noise, filter or use median read.

CALIBRATION & TESTING PLAN

- **Sensor calibration**
- Measure known DC voltages (multimeter) and compute ADC-to-voltage factor for the divider.
- Pass known currents (use adjustable load or bench supply + ammeter) and

  calibrate ACS712 sensitivity and zero-offset.

- **Tracker tuning**
- In sunlight, note LDR raw readings left/right at different sun angles; set LDR_DIFF_THRESHOLD so noise doesn't cause movement.
- Adjust STEP so servo moves smoothly. Verify servo limits to prevent mechanical strain.

- **Functional tests**
- Compare energy harvested over a controlled time (e.g., 1 hour) with tracker

OFF (panel fixed) and tracker ON. Log readings to ThingSpeak and capture comparison charts.

- Simulate shading (cover part of panel) and confirm tracker reorients.

 Network tests

- Verify ThingSpeak uploads at set interval; check success codes and handle failed uploads gracefully.

 Safety checks

- Check for loose wiring, secure servo mount, and ensure no part overheats.

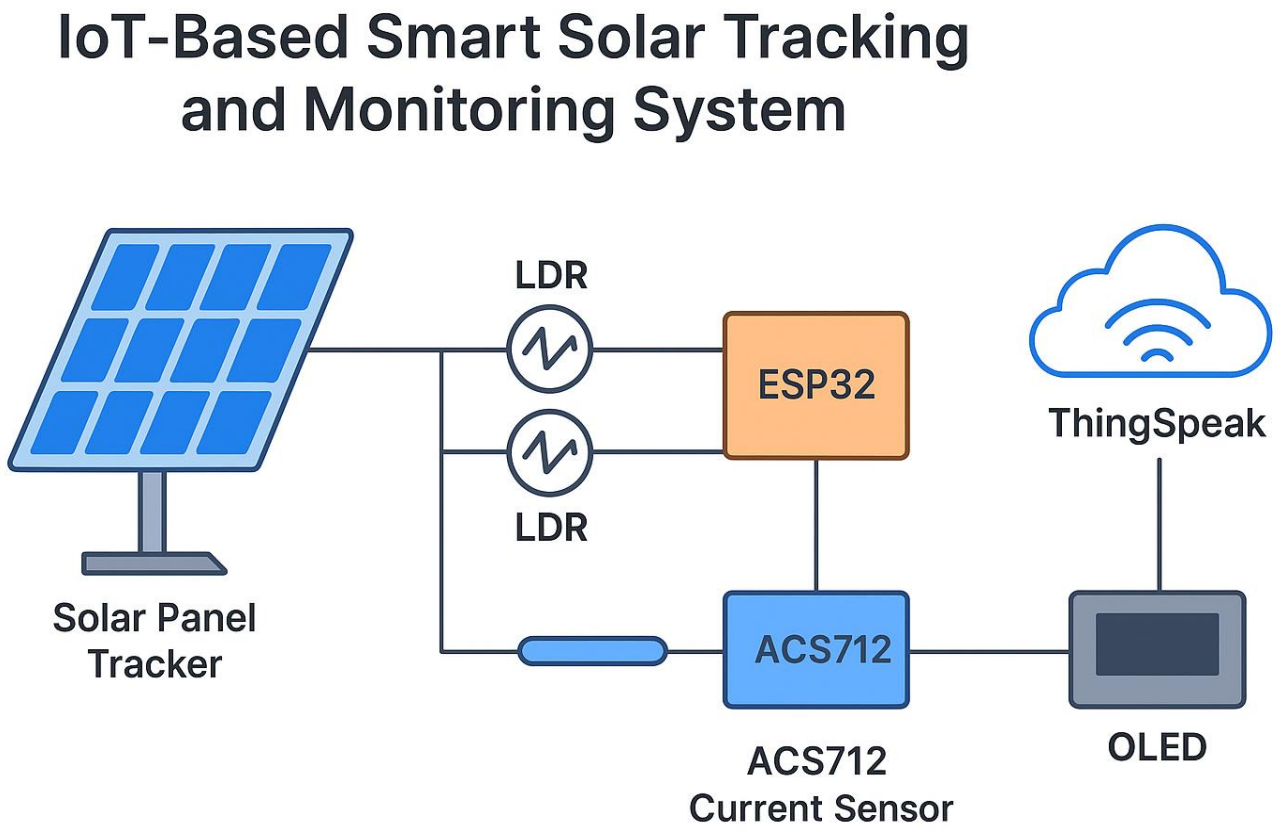
DISCUSSION & LIMITATIONS

- **Accuracy:** Low-cost ACS712 and ESP32 ADC give approximate current and voltage; expect measurement errors (calibrate to improve).
- **Servos & mechanical limits:** Micro servos suit small panels only; larger panels need geared motors and limit switches.
- **Power cost vs gain:** For small panels the power consumed by servos and electronics may offset gains; quantify this in report by measuring system own consumption.
- **Weather & reliability:** Wind/load can disturb alignment; consider locking mechanism or wind cut-off.

FUTURE IMPROVEMENTS

- Replace ACS712 + ADC with **INA219** or dedicated energy metering IC (INA219 gives I2C current+power with better accuracy).
- Add **MPPT** (perturb & observe) algorithm to maximize output with DC-DC converter.
- Use **geared stepper motors** with position encoders for robust tracking of larger panels.
- Add historical data storage (SD card) and analytics (Grafana).
- Implement power budget: disable tracking at night or during very low irradiance to save servo power.

SAFETY NOTES

- Work only with **low-voltage** panels for prototyping (<30V). If higher voltages are used, use proper isolation and safety gear.
- Use fuses and Schottky diodes to prevent reverse currents.
- Keep wiring tidy and secure servo mounts to avoid mechanical failure.
- Calibration procedure and sample data (tables + ThingSpeak screenshots).
- Discussion, limitations, conclusion, references (datasheets)

BLOCK DIAGRAM